GUIA VISUAL 054

READ, ECHO E PRINTF

O relatório saía com três colunas alinhadas na máquina de quem escreveu. Chamado pelo `cron` com `sh inventario.sh`, virou `-e Servidor\tCPU\tMemoria`. O culpado é o `echo` — o único comando do shell cuja portabilidade é problema real.

054
GUIAS LINUX

O ECHO MENTE · O PRINTF NÃO

`echo -e "a\tb"`
Três implementações discordam: o **dash** (o `/bin/sh` do Kali) *sempre* interpreta a barra e imprime o `-e` literal; o **Bash** faz o inverso; com `shopt -s xpg_echo`, o Bash vira dash.

`valor="-n"; echo "$valor"`
Imprime **nada** — o valor virou opção. Qualquer valor vindo de fora que comece com hífen é engolido sem aviso.

`printf "$entrada"`
Dado **nunca** entra na cadeia de formato: um `%d%d%d` no valor consome argumentos que não existem. O certo é `printf '%s\n' "$entrada"`.

`printf '%b\n' "$txt"`
O substituto seguro de `echo -e`: interpreta a barra invertida do *argumento*, não do formato.

`%` `-` `10` `.2` `f`

`-` `sinalizador` — alinha à esquerda (padrão: direita)
`10` `largura` — ocupa 10 colunas, completa com espaço
`.2` `precisão` — 2 casas decimais
`f` `conversor` — ponto flutuante

A cadeia é reaproveitada ciclicamente até acabarem os argumentos: `printf '%-8s %5.1f\n' raiz 42.75 var 8.5 home 120.4` imprime **três linhas** sem laço nenhum — e é por isso que `printf '%s\n' "$@"` põe cada argumento na sua linha.
`%s` texto · `%d` inteiro · `%.2f` decimal · `%x` hex · `%o` octal (permissões!) · `%b` com barras · `%q` escapado para o shell · `%%` literal.

O IDIOMA CANÔNICO DO LAÇO DE LEITURA — DECORE O BLOCO INTEIRO

`while` `IFS=` `read` `-r` `linha` `|| [[ -n "$linha" ]]` `; do … done` `< "$arq"`

`IFS=`
Esvazia o separador **só nesta linha**. Sem ele, o `read` apara espaços e tabulações das pontas — e a linha indentada perde a indentação.

`-r`
Desliga o tratamento da barra invertida. Sem ele, `C:\Users` vira `C:Users` e uma linha terminada em `\` é colada na seguinte.

`|| [[ -n "$linha" ]]`
Salva a **última linha** quando o arquivo não termina em quebra — caso em que o `read` lê o conteúdo e mesmo assim retorna falha.

`< "$arq"`
Redirecionamento, não *pipe*: `cat arq | while` funciona, mas cria **subshell** e as variáveis alteradas no laço voltam vazias.

As três formas erradas mais comuns: `for linha in $(cat arq)` quebra em **espaços**, não em linhas · `cat arq | while read linha` subshell + perde indentação · `while read linha` come barra invertida e espaços. E para dividir campos, o `IFS` trabalha a favor: `while IFS=: read -r usuario _ uid _ _ _ shell; do … done < /etc/passwd` — o `IFS=:` vale só para aquele `read`, o do script fica intacto.

kali@kali: ~/lab-io

```
$ erro() { printf 'ERRO: %s\n' "$*" >&2; }   # erro no descritor 2: o usuario
$ aviso(){ printf 'aviso: %s\n' "$*" >&2; }  # redireciona a saida util e ainda ve
$ [[ -t 1 ]] && V="$(tput setaf 2)" N="$(tput sgr0)" || { V=; N=; }
# sem o teste -t 1, redirecionar grava ^[[32m cru no arquivo; e no cron o TERM nem existe
$ printf '%-18s %8s %10s\n' SERVICO ESTADO PID   # tabela: largura fixa alinha
$ while IFS= read -r host; do ssh -n "$host" uptime; done < alvos.txt
# o -n e OBRIGATORIO: sem ele o ssh le o resto do arquivo e o laco roda 1 vez so
# vale para ssh, ffmpeg, mysql, apt. Alternativa: read -u 3 ... done 3< alvos.txt
```

Senha lida com `read -s` continua exposta. O `-s` resolve o ombro do colega e mais nada: o valor fica numa variável do shell (legível pelo root), **aparece inteiro no rastreio se o script rodar com `set -x`**, e some para o `ps aux` de qualquer usuário se você o passar adiante como argumento. O caminho correto: `read -rs` seguido de entrada padrão — `printf '%s' "$senha" | comando --password-stdin` — ou um gerenciador como o `pass`; e `set +x` antes de qualquer linha que toque na variável. Ao confirmar ação irreversível, o padrão é **não**: `read -rp "Apagar? [s/N] " r` com `${r,,}` para comparar em minúsculas, e `rm -rf -- "$ALVO"` — o `--` encerra as opções e protege contra caminho iniciado por hífen. Sem terminal (`[[ ! -t 0 ]]`), aborte com mensagem: no `cron`, um `read` sem `-t` espera para sempre.

RESUMO RÁPIDO

- ✓ **`echo` só para texto literal simples.** Bash, dash e `/bin/echo` discordam sobre `-e`, e valor começado com hífen vira opção.
- ✓ **`printf` é o substituto em qualquer shell**, e a cadeia se reaproveita: `printf '%s\n' "$@"` e as tabelas com `%-20s` saem em uma linha.
- ✓ **Dado nunca entra na cadeia de formato** — sempre `printf '%s\n' "$var"`.
- ✓ **Erro vai para o descritor 2** (`>&2`) e **cor só com `[[ -t 1 ]]`**, senão o arquivo recebe códigos de escape crus.
- ✓ **`read` sem `-r` corrompe dados** e sem `IFS=` apara as pontas. As duas são obrigatórias, e nenhuma substitui a outra.
- ✓ **Comando de dentro do laço pode comer a entrada do laço:** `ssh -n`, `< /dev/null` ou um descritor separado.

NO KALI

Lista de alvos é o uso nº 1 do `read` em laboratório — e o que mais quebra. Um `alvos.txt` real tem linha em branco, comentário com `#`, espaço no fim e, se veio de planilha, terminador **CRLF**. Limpe no começo do laço: `linha="${linha%$'\r'}"` e `[[ -z "${linha// }" || "$linha" == \#* ]] && continue`. Apontar a ferramenta para um alvo com um retorno de carro colado no nome é como um teste autorizado vira acesso não autorizado — e a Lei 12.737/2012 não distingue intenção de descuido; valide e exiba a lista antes de agir. **O `tput` falha em muito contexto do Kali**: contêiner sem `ncurses-base`, `screen` mal configurado, shell reverso — `TERM` ausente ou `dumb`. O par `[[ -t 1 ]] + tput … 2>/dev/null` deixa o script em preto e branco em vez de encher o relatório de erro. Para script de configuração com várias perguntas, `whiptail --yesno/--inputbox/--menu` já está instalado (é o que o `kali-tweaks` usa) — devolve o valor pela saída de **erro**, daí a inversão `3>&1 1>&2 2>&3`. E relatório que outra ferramenta vai ler não se alinha com espaços: `printf '%s\t%s\t%s\n'` ou JSON linha a linha, para o `jq` e o `awk` trabalharem sem heurística.